

# Technical Architecture of Deterministic Web-Based Multiplayer Systems: A Deep Analysis of WebRTC, Temporal Accumulators, and WebAssembly Memory Persistence

The evolution of the web from a collection of static documents to a robust platform for high-performance, real-time applications has necessitated a fundamental rethinking of network protocols and execution models. In the context of competitive multiplayer web games, the shift toward deterministic lockstep architectures is driven by the need for synchronization across heterogeneous environments while navigating the inherent constraints of the browser's security sandbox and the unpredictability of the public internet. Achieving a seamless experience requires an intricate synthesis of three primary technological pillars: ultra-low-latency transport via WebRTC Data Channels, consistent temporal execution through fixed-step accumulators, and instantaneous state recovery using WebAssembly (WASM) linear memory serialization. This technical analysis explores the mechanisms, algorithmic trade-offs, and implementation strategies required to build production-grade multiplayer systems on the web.

## Transport Layer Architectures: WebRTC vs. WebSockets

The primary architectural decision in any networked application is the choice of transport protocol. For the web, this choice has historically been restricted to the Transmission Control Protocol (TCP) through WebSockets, but the maturation of WebRTC (Web Real-Time Communication) has introduced the possibility of User Datagram Protocol (UDP) interactions, which are essential for minimizing latency in twitch-based genres.

### WebSockets: The Constraint of Sequential Reliability

WebSockets, established in 2011 as RFC 6455, provide a persistent, full-duplex communication channel over a single TCP connection. The protocol begins with an HTTP/1.1 handshake using an "Upgrade" header, which, upon server acceptance, transitions the connection into a binary-friendly stream that eliminates the overhead of repeated HTTP headers. While WebSockets are highly effective for server-mediated scenarios like real-time chat, collaborative editing (e.g., Google Docs), and turn-based strategy games, they suffer from inherent limitations in competitive environments.

The fundamental issue with WebSockets is the Head-of-Line (HoL) blocking inherent to TCP. TCP is designed for guaranteed, ordered delivery; if a single packet is lost due to network congestion, the protocol halts the delivery of all subsequent packets until the lost data is retransmitted and acknowledged. In a deterministic lockstep game, where the simulation cannot advance until the inputs for all players have arrived, this retransmission delay causes the entire match to "stutter" or freeze. Consequently, while WebSockets offer 99%+ browser support and a simpler implementation model, they introduce a "reliability tax" that manifests as spikes in latency (jitter) under poor network conditions.

## WebRTC Data Channels: The UDP-Based Alternative

WebRTC Data Channels offer a more granular approach to networking by utilizing the Stream Control Transmission Protocol (SCTP) over Datagram Transport Layer Security (DTLS) over UDP. Unlike WebSockets, WebRTC is primarily designed for peer-to-peer (P2P) communication, allowing data to travel directly between browsers without the mediation of a central server, which significantly reduces the round-trip time (RTT).

WebRTC Data Channels are highly configurable, allowing developers to choose between reliable/ordered delivery (mimicking TCP) and unreliable/unordered delivery (mimicking raw UDP). For real-time multiplayer, the unreliable/unordered mode is the gold standard, as it allows the game to discard stale packets (e.g., a player's position from three ticks ago) in favor of the most recent data, thereby avoiding the HoL blocking that plagues WebSockets.

However, this performance comes at the cost of significantly higher implementation complexity. To establish a WebRTC connection, peers must use a signaling server—often implemented via WebSockets—to exchange Session Description Protocol (SDP) offers and answers, as well as Interactive Connectivity Establishment (ICE) candidates for NAT traversal. Furthermore, if direct P2P connectivity fails due to strict firewalls (common in corporate or educational networks), the system must fall back to a Traversal Using Relays around NAT (TURN) server, which acts as a relay and adds to the infrastructure cost.

## Comparative Technical Metrics of Transport Protocols

The trade-offs between these two protocols can be quantified through several key metrics, including header overhead, setup complexity, and performance under packet loss.

| Feature         | WebSockets         | WebRTC Data Channel                |
|-----------------|--------------------|------------------------------------|
| Transport Layer | TCP                | UDP                                |
| Delivery Mode   | Reliable / Ordered | Configurable (Reliable/Unreliable) |

|                           |                        |                                  |
|---------------------------|------------------------|----------------------------------|
| <b>Congestion Control</b> | TCP Standard           | SCTP-based                       |
| <b>NAT Traversal</b>      | Not required           | STUN/TURN/ICE required           |
| <b>P2P Support</b>        | No (Client-Server)     | Yes (Native)                     |
| <b>Header Overhead</b>    | ~2–14 bytes            | ~120 bytes<br>(IP+UDP+DTLS+SCTP) |
| <b>Scalability</b>        | High (Server-mediated) | Low for Mesh (Scales via SFU)    |
| <b>Encryption</b>         | Optional (WSS)         | Mandatory (DTLS/SRTP)            |

For developers targeting competitive gaming, the ~120-byte overhead of WebRTC is generally acceptable because it provides sub-50ms latency and eliminates retransmission-induced judder. Recent benchmarks on modern browser implementations show that while local throughput can exceed 18 MB/s, wide-area performance often suffers because the default SCTP window size is not dynamically adjusted for high-latency links.

# Temporal Accumulators and Fixed-Step Simulation

A cornerstone of deterministic multiplayer is the requirement that all clients execute the game simulation at the exact same rate. If the simulation logic were tied to the rendering frame rate—which varies based on the user's hardware (e.g., 60Hz vs. 144Hz monitors) and CPU load—the game would quickly desynchronize.

## The Accumulator Pattern

The "fixed-step temporal accumulator" is the standard solution to this problem. This algorithm decouples the simulation ticks from the rendering frames by maintaining a "bank" of time (the accumulator). In each rendering loop—typically triggered by `requestAnimationFrame`—the elapsed time since the previous frame is added to the accumulator. The simulation is then advanced in discrete increments (e.g., 16.66ms for a 60Hz simulation) as many times as possible until the accumulator is depleted.

The implementation follows this general structure in LaTeX notation to represent the discrete time steps:

Let  $t_{current}$  be the time of the current frame and  $t_{prev}$  be the time of the previous frame. The delta time is  $\Delta t = t_{current} - t_{prev}$ . The accumulator is updated as:  $A = A + \Delta t$ . The

simulation advances while  $A \geq dt_{fixed}$  :

Simulate( $dt_{fixed}$ )

$$A = A - dt_{fixed}$$

This ensures that the simulation logic always uses a constant  $dt_{fixed}$ , which is essential for deterministic physics and consistent character movement across different machines. If a frame takes 100ms to render due to a CPU spike, the while loop will execute 6 times to ensure the simulation stays caught up with real time.

## Visual Interpolation

Because the rendering cycle and the simulation cycle are decoupled, the renderer will often be asked to draw the game at a time that falls between two simulation ticks. This can cause visual "jitter" because the objects appear to jump between fixed points. The solution is to calculate a blending factor  $\alpha$ , representing the fraction of the current simulation step remaining in the accumulator:

$$\alpha = \frac{A}{dt_{fixed}}$$

The rendering engine then interpolates the position of objects between their previous state ( $S_{i-1}$ ) and their current state ( $S_i$ ):

$$P_{render} = S_{i-1} \cdot (1 - \alpha) + S_i \cdot \alpha$$

This technique provides the visual smoothness of variable-rate rendering while preserving the underlying mathematical stability of a fixed-rate simulation.

## Latency Mitigation: Jitter Buffering and Input Delay Algorithms

In a networked environment, the "Global Tick" is governed by the server or a designated host. However, network latency and jitter (the variance in that latency) make it impossible for clients to have a perfectly synchronized view of time without sophisticated buffering and clock sync algorithms.

## Clock Synchronization and Offset Estimation

Before a match can begin, clients must synchronize their local clocks with the server's time reference. Cristian's Algorithm is a common probabilistic method for this purpose. The client sends a request at  $t_0$ , the server responds with its current time  $T_{server}$ , and the client receives the response at  $t_1$ . Assuming symmetric network delay, the round-trip time is  $RTT = t_1 - t_0$ , and the estimated one-way delay is  $D = RTT/2$ . The client then calculates its clock offset as:

$$\text{Offset} = T_{server} + D - t_1$$

By taking multiple samples and using the median or a weighted average that discards outliers (a technique inspired by Marzullo's Algorithm), the client can establish a robust correlation between local and server time. Accuracy within 1–10ms is typically achievable in the browser.

## Adaptive Input Delay Buffers

Deterministic lockstep (or "Input Delay") netcode works by having all clients send their button presses to the server and waiting until those inputs arrive at every other client before executing the simulation frame. This introduces a "sluggish" feel, as there is a delay between a keypress and the on-screen action.

To mitigate this while preventing "stutter" (where the simulation stops because a packet arrived late), developers use an adaptive jitter buffer. This buffer monitors the network conditions and dynamically adjusts the "input delay". If the network jitter—calculated using the Van Jacobson variance formula—is high, the buffer increases the delay to ensure all packets arrive in time for their scheduled tick. If the network stabilizes, the buffer slowly "decays" and reduces the delay to improve responsiveness.

The variance formula for jitter estimation used in RFC 793 and by Van Jacobson is as follows:

Let  $n_i$  be the one-way delay of the  $i$ -th packet. The smoothed delay  $d_i$  and the variation  $v_i$  are calculated as:

$$d_i = (1 - \alpha) \cdot d_{i-1} + \alpha \cdot n_i$$

$$v_i = (1 - \beta) \cdot v_{i-1} + \beta \cdot |n_i - d_i|$$

Where  $\alpha = 0.125$  and  $\beta = 0.25$  are common weighting factors for an Exponentially

Weighted Moving Average (EWMA).

## Simulation Speed Scaling

A more advanced technique for stutter mitigation is simulation speed scaling. Instead of simply pausing the game when the buffer is empty (an under-run), the game can slightly slow down the simulation (e.g., by 5%) if the buffer is running low, allowing late packets more time to arrive. Conversely, if the buffer is becoming too full, the simulation can be sped up (e.g., by 5%) to "catch up" to the server's time. Because these speed adjustments are minute and applied consistently across all deterministic simulations, they are often imperceptible to the player while preventing jarring freezes.

| Network Metric | Threshold   | Recommended Action                     |
|----------------|-------------|----------------------------------------|
| Ping (RTT)     | < 50ms      | Low input delay (1–2 frames)           |
| Ping (RTT)     | 100ms–200ms | Rollback netcode / High prediction     |
| Jitter         | < 5ms       | Minimize buffer size                   |
| Jitter         | > 20ms      | Increase buffer / Enable speed scaling |
| Packet Loss    | > 2%        | Reliable fallback / FEC / Redundancy   |

## Network Congestion Control in WebRTC

WebRTC includes built-in congestion control algorithms like Google Congestion Control (GCC), which monitor packet loss and RTT to dynamically adjust the bitrate of media streams. For the Data Channel, SCTP handles its own flow control and congestion avoidance. However, developers must still monitor the `bufferedAmount` property of the `RTCDataChannel` to avoid memory bloat when the application attempts to send data faster than the network can transmit it.

## Rollback Netcode: The Predictive Evolution

For high-speed genres like fighting games (e.g., *Street Fighter*) and shooters, input delay alone is insufficient because players react within frames. Rollback netcode, exemplified by the GGPO (Good Game Peace Out) SDK, represents a paradigm shift in latency mitigation.

In a rollback model, the local client applies player inputs immediately, providing zero-latency

feedback. Simultaneously, the client predicts the inputs of remote players—typically by assuming their current state will remain unchanged (e.g., they continue holding "Left"). When the actual inputs arrive from the network several frames later, the client compares them to the predictions. If the predictions were wrong, the game engine "rolls back" its state to the frame where the discrepancy occurred, applies the correct inputs, and re-simulates all frames up to the present within a single rendering cycle.

This process relies heavily on two factors:

- **High-Performance Re-simulation:** The simulation logic must be extremely fast, as the client may need to simulate 10 or more frames in a few milliseconds.
- **Instant State Restoration:** The ability to "save" and "load" the entire game state at any frame is critical.

## WebAssembly and Mid-Session State Recovery

WebAssembly (WASM) has become the de facto execution engine for deterministic games on the web due to its near-native performance and predictable floating-point arithmetic. One of its most powerful features for multiplayer is the "linear memory" model—a contiguous, monolithic array of bytes that contains the entire program heap and stack.

### Linear Memory Serialization

In traditional JavaScript engines, serializing game state is difficult because data is scattered across numerous high-level objects and managed by a garbage collector. In WASM, the entire state is contained in a single `ArrayBuffer`. Snapshotting the game's state for a rollback or for a late-joining client is as simple as performing a `memcpy` or a `slice()` on this buffer.

Restoring the state is equally efficient:

Because this is a contiguous slice of bytes with no pointers that point "outside" the linear memory, the restoration is perfectly faithful to the original state. This "black box" approach allows developers to treat the game logic as an opaque state machine, making it easier to integrate complex physics engines (like `Ammo.js`) or scripting languages (like `Lua`) that were not originally designed for networking.

### Late-Joining and Persistence Mechanics

A common challenge in P2P multiplayer is the "Late-Join" problem: how to bring a new client into a game already at Tick 10,000 without requiring them to replay the entire history.

WASM memory serialization provides a streamlined path for mid-session recovery:

- **State Request:** The joining client connects via WebRTC and requests the current state from a peer.
- **Binary Snapshot:** The peer takes a full snapshot of its WASM linear memory and transmits

it over a reliable WebRTC Data Channel.

- **Instantiation:** The joining client receives the binary blob, initializes its own WASM instance, and "injects" the memory snapshot into the module.
- **Synchronization:** The joining client then begins receiving real-time input packets to align its simulation with the rest of the group.

## Delta Compression for Large State Transfer

As games grow in complexity, the WASM linear memory can reach hundreds of megabytes or even gigabytes (with the advent of WASM 3.0's 64-bit address space). Transferring a full snapshot of this size to a late-joiner would consume excessive bandwidth and time.

To solve this, developers employ delta synchronization. Instead of sending the entire memory, the host compares the current memory state to a previous reference state (e.g., from 10 seconds ago) and identifies the "dirty" pages. Only the changed bits—often compressed using an XOR binary diff or algorithms like zstd—are transmitted. This "WASMrsync" approach significantly cuts the computation burden on the client and reduces rejoin times by up to 95% in high-latency environments.

## Determinism Drift and Floating-Point Consistency

A critical hurdle in using WASM for deterministic lockstep is "cross-platform determinism drift". While integer arithmetic is universally consistent, floating-point operations can diverge slightly across different CPU architectures (e.g., x86 vs. ARM) due to differences in NaN (Not-a-Number) bit patterns or SIMD (Single Instruction, Multiple Data) optimizations.

To prevent desynchronization, web developers often use "fixed-point" math libraries or ensure that their WASM toolchain (like Emscripten) is configured to prioritize IEEE 754 compliance over raw performance. Furthermore, standard library implementations of functions like `std::mt19937` (Mersenne Twister) can vary depending on whether the target architecture is 32-bit or 64-bit, necessitating a custom, architecture-agnostic random number generator.

## Practical Implementation: A Synthesis of Networking and State

When building a real-world system, the interaction between these layers must be carefully managed to avoid performance bottlenecks.

### The JavaScript-WASM Boundary

One of the most significant performance drains in browser-based games is the "context-switching" cost between the JavaScript environment and the WebAssembly runtime. Every time a JS function calls a WASM function (or vice-versa), there is a performance penalty.

To optimize this, advanced architectures treat the JS/WASM boundary as a high-latency



network. Instead of making hundreds of tiny calls (e.g., to update individual entity positions), the system batches instructions in WASM and sends a single "flush" signal to JS. The JS side then reads all rendering data directly from a shared region of WASM memory in one go. This batching approach can increase the frame rate from 40 FPS to over 100 FPS in entity-heavy games.

## Memory Allocation Strategies for Snapshotting

Efficient snapshotting requires a specialized memory allocator within WASM. A "SnapshottableAllocator" divides memory into distinct sections:

- **Volatile State:** Contains the game heap (entities, physics, variables). This is the only part copied during snapshots.
- **Static Assets:** Contains textures, sounds, and constant data. This is initialized at startup and never snapshotted, saving megabytes of redundant data copy.
- **Scratch Memory:** Used for temporary calculations that do not need to persist across frames.

By ensuring the volatile state is stored at a fixed memory offset (e.g., 0x800000000), the developer ensures that internal pointers remain valid when the memory is restored.

## Security and Cheating in P2P Lockstep

A major drawback of the peer-to-peer lockstep model is its vulnerability to cheating. Because every client is an "authority" on the game state and receives the inputs of all other players, a malicious user could modify their local client to see through walls or manipulate their own inputs.

Mitigation strategies include:

- **Input Validation:** Each client validates that remote inputs are "legal" within the game's rules.
- **State Hash Verification:** Peers periodically exchange a hash (checksum) of their WASM linear memory. If a hash mismatch is detected, the game is desynchronized and terminated.
- **Authoritative Relay:** In high-stakes environments, a central "Headless" server (also running the WASM simulation) acts as the final arbiter, verifying all inputs and memory hashes before broadcasting them.

## Conclusion: Future Directions in Web Multiplayer

The convergence of WebRTC Data Channels, fixed-step accumulators, and WASM memory serialization has unlocked a new era of competitive web gaming that was previously impossible. The ability to achieve near-native performance while leveraging the web's universal accessibility allows for frictionless multiplayer experiences that bridge the gap between

platforms.

Looking forward, several technologies are poised to further refine this stack:

- **WebTransport:** A new API providing a low-latency, bidirectional, multiplexed transport protocol over HTTP/3 (QUIC). This may eventually offer a simpler alternative to WebRTC for client-server architectures by providing UDP-like unreliable delivery without the complexity of ICE/STUN/TURN.
- **WASM 3.0 and Beyond:** Features like "Multi-Memory" and "Wasm GC" will allow for more sophisticated memory management and better integration with high-level languages like Java and Python.
- **Edge Computing:** Deploying WASM-based "Headless" authoritative servers at the network edge (via platforms like Cloudflare Workers or Edgegap) will minimize the physical distance data must travel, bringing RTTs closer to the physical limits of light-in-fiber (~5ms per 1,000 km).

By mastering the intricate dance between network protocols, temporal logic, and memory structures, developers can create web-based titles that offer the same responsiveness and depth as native desktop applications, fully realizing the potential of the browser as a premier gaming platform.

## References

- WebRTC vs WebSocket Explained: When to Use What (A Real-World Story), <https://dev.to/abdullahmubin/webrtc-vs-websocket-explained-when-to-use-what-a-real-world-story-5de9>
- WebSocket vs WebRTC: When to Use Each Protocol, <https://websocket.org/comparisons/webrtc/>
- WebRTC vs. WebSockets for multiplayer games - Rune, <https://developers.rune.ai/blog/webrtc-vs-websockets-for-multiplayer-games>
- WebRTC vs. WebSocket: Choosing Your Real-Time Communication Protocol, <https://sheerbit.com/choosing-between-webrtc-and-websocket-a-simplified-guide/>
- WebRTC vs WebSockets: What Are the Differences? - GetStream.io, <https://getstream.io/blog/webrtc-websockets/>
- WebRTC vs WebSockets: Complete 2026 Comparison Guide - Ant Media Server, <https://antmedia.io/webrtc-vs-websockets-what-are-the-differences/>
- Best WebSocket Libraries + Benchmarks - PieHost, <https://piehost.com/websocket/best-websocket-libraries-benchmarks>
- How to Mitigate Latency in Multiplayer Games: Input Delay vs Rollback - Edgegap, <https://edgegap.com/blog/how-to-mitigate-latency-in-multiplayer-games-input-delay-vs-rollback>
- WebRTC vs WebSocket: A Benchmark Study - Tragofone, <https://tragofone.com/webrtc-vs-websocket-real-time-communication-comparison/>
- WebRTC Data Channels - 4 layer cake of protocols - YouTube,

<https://www.youtube.com/watch?v=ZkEOSrHEXX8>

- WebRTC Data Channels: A guide. - Metered,  
<https://www.metered.ca/blog/webrtc-data-channels-a-guide/>
- Understanding the WebRTC Protocol - VoIPmonitor.org,  
[https://www.voipmonitor.org/doc/Understanding\\_the\\_WebRTC\\_Protocol](https://www.voipmonitor.org/doc/Understanding_the_WebRTC_Protocol)
- An architectural overview for WebRTC — A protocol for implementing video conferencing,  
<https://eytanmanor.medium.com/an-architectural-overview-for-web-rtc-a-protocol-for-implementing-video-conferencing-e2a914628d0e>
- WebRTC vs WebSockets: When to Use Each for Real-Time Apps - Vijay T - Technical Blog,  
<https://blog.vijayt.com/webrtc-vs-websockets-when-to-use-each-for-real-time-apps/>
- WebRTC vs. WebSocket: Key differences and which to use - Ably Realtime,  
<https://ably.com/topic/webrtc-vs-websocket>
- WebRTC for Browser Based Multiplayer Games Building Real Time Gaming Experiences,  
<https://www.daydreamsoft.com/blog/webrtc-for-browser-based-multiplayer-games-building-real-time-gaming-experiences>
- An Open-Source Framework Using WebRTC for Online Multiplayer Gaming - ResearchGate,  
[https://www.researchgate.net/publication/375689194\\_An\\_Open-Source\\_Framework\\_Using\\_WebRTC\\_for\\_Online\\_Multiplayer\\_Gaming](https://www.researchgate.net/publication/375689194_An_Open-Source_Framework_Using_WebRTC_for_Online_Multiplayer_Gaming)
- WebRTC Overhead - Stack Overflow,  
<https://stackoverflow.com/questions/11934499/webrtc-overhead>
- WebRTC Architecture Basics: P2P, SFU, MCU, and Hybrid Approaches - Medium,  
<https://medium.com/securemeeting/webrtc-architecture-basics-p2p-sfu-mcu-and-hybrid-approaches-6e7d77a46a66>
- Architecting WebRTC for Surveillance and Remote Monitoring - Wowza,  
<https://www.wowza.com/blog/architecting-webrtc-for-surveillance-and-remote-monitoring>
- Performance Evaluation of WebRTC Data Channels,  
[https://tuhat.helsinki.fi/ws/portalfiles/portal/167373638/Eskola\\_webrtc.pdf](https://tuhat.helsinki.fi/ws/portalfiles/portal/167373638/Eskola_webrtc.pdf)
- Performance Evaluation of WebRTC Data Channels - VTT's Research Information Portal,  
<https://cris.vtt.fi/en/publications/performance-evaluation-of-webrtc-data-channels/>
- Window: requestAnimationFrame() method - Web APIs | MDN,  
<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>
- Time-Based Animation in Javascript using RequestAnimationFrame - NickNagel.com,  
<https://dr-nick-nagel.github.io/blog/raf-time.html>
- game-mechanics-optimizations/06\_fixed\_timestep.md at main - GitHub,  
[https://github.com/raduacg/game-mechanics-optimizations/blob/main/06\\_fixed\\_timestep.md](https://github.com/raduacg/game-mechanics-optimizations/blob/main/06_fixed_timestep.md)
- Why use a variable delta time in a physics game loop rather than a fixed time step? - Reddit,  
[https://www.reddit.com/r/gamedev/comments/uktah1/why\\_use\\_a\\_variable\\_delta\\_time\\_in\\_a\\_physics\\_game/](https://www.reddit.com/r/gamedev/comments/uktah1/why_use_a_variable_delta_time_in_a_physics_game/)
- Gaffer on Games: Fix your timestep (Question) : r/gamedev - Reddit,

[https://www.reddit.com/r/gamedev/comments/10bs23m/gaffer\\_on\\_games\\_fix\\_your\\_timestep\\_question/](https://www.reddit.com/r/gamedev/comments/10bs23m/gaffer_on_games_fix_your_timestep_question/)

- Gaffer on Games | Fix Your Timestep!, <http://vodacek.zvb.cz/archiv/681.html>
- Fixed timestep jittering : r/gamedev - Reddit, [https://www.reddit.com/r/gamedev/comments/t7mog5/fixed\\_timestep\\_jittering/](https://www.reddit.com/r/gamedev/comments/t7mog5/fixed_timestep_jittering/)
- What is jitter variance in networking? - Stack Overflow, <https://stackoverflow.com/questions/58463073/what-is-jitter-variance-in-networking>
- Cristian's algorithm - Wikipedia, [https://en.wikipedia.org/wiki/Cristian%27s\\_algorithm](https://en.wikipedia.org/wiki/Cristian%27s_algorithm)
- Game Networking (2) - Time, Tick, Clock Synchronisation | by Daposto | Medium, <https://daposto.medium.com/game-networking-2-time-tick-clock-synchronisation-9a0e76101fe5>
- Clock Synchronization Is a Nightmare - Arpit Bhayani, <https://arpitbhayani.me/blogs/clock-sync-nightmare/>
- Sync server time to client browser with JS. Implement follow Network Time Protocol. - GitHub Gist, <https://gist.github.com/6abb1d437dbe406fbed6>
- Clock synchronization, [https://www.cs.put.poznan.pl/akobusinska/downloads/rso/ME\\_wyklad\\_Profesor.pdf](https://www.cs.put.poznan.pl/akobusinska/downloads/rso/ME_wyklad_Profesor.pdf)
- Distributed Clock Synchronization Algorithms: A Survey, [https://serialsjournals.com/abstract/26854\\_23.pdf](https://serialsjournals.com/abstract/26854_23.pdf)
- How to sync clocks over networking for game development? - Software Engineering Stack Exchange, <https://softwareengineering.stackexchange.com/questions/201756/how-to-sync-clocks-over-networking-for-game-development>
- Determinism, Prediction and Rollback - coherence Documentation, <https://docs.coherence.io/manual/advanced-topics/competitive-games/determinism-prediction-rollback>
- 赵博强/ggpo - Gitee, <https://gitee.com/zhaoboqiang/ggpo>
- US20100296525A1 - Simple Adaptive Jitter Buffering Algorithm For Network Nodes - Google Patents, <https://patents.google.com/patent/US20100296525A1/en>
- Jitter Buffer - What is it and how does it work? - GetStream.io, <https://getstream.io/glossary/jitter-buffer/>
- Jitter Buffer: Optimising Real-Time Communications - Digital Samba, <https://www.digitalsamba.com/blog/jitter-buffers-optimising-real-time-communications>
- Adaptive Buffering - TCP Streamer - Mintlify, <https://mintlify.com/NaturalDevCR/TCP-Streamer/features/adaptive-buffering>
- An Adaptive Jitter Buffer Playout Algorithm for Enhanced VoIP Performance - Suman Bhunia, <https://sbhunias.me/publications/manuscripts/acity11.pdf>
- yingwang/adaptive-jitter-buffer - GitHub, <https://github.com/yingwang/adaptive-jitter-buffer>
- The Ultimate Gaming Latency Guide: How to Eliminate Lag and Maximize FPS, <https://www.worldstream.com/es/gaming-latency-guide/>
- Optimizing Online Gaming Performance: The Impact of DSL Interleave Modes on Latency - IEEE Xplore, <https://ieeexplore.ieee.org/iel8/6287639/11323511/11314184.pdf>

- Server-side rewind | Netcode for Entities | 1.7.0 - Unity - Manual, <https://docs.unity3d.com/Packages/com.unity.netcode@1.7/manual/server-rewind.html>
- Low-Latency WebRTC Streaming: Real-Time Video at Scale - Flussonic, <https://flussonic.com/blog/article/low-latency-webrtc-streaming>
- How to Fix Input Lag - Intel, <https://www.intel.com/content/www/us/en/gaming/resources/how-to-fix-input-lag.html>
- The lag-fighting techniques behind GGPO's netcode - Game Developer, <https://www.gamedeveloper.com/programming/the-lag-fighting-techniques-behind-ggpo-s-netcode>
- WASM For Core Game Loop & Mods · fishfolk jumpy · Discussion ..., <https://github.com/fishfolk/jumpy/discussions/489>
- Jfgeyelin's dev blog, <https://www.jfgeyelin.com/>
- wasmbox - Rust - Docs.rs, <https://docs.rs/wasmbox>
- Memory Efficient WebAssembly Containers - Atlarge Research, <https://atlarge-research.com/pdfs/mjansen-wasm-2025.pdf>
- GitHub - rameshvarun/netplayjs: Make P2P multiplayer browser games, no server hosting or synchronization code required. Powered by rollback netcode + WebRTC., <https://github.com/rameshvarun/netplayjs>
- WebAssembly Linear Memory - Wasm By Example, <https://wasmbyeexample.dev/examples/webassembly-linear-memory/webassembly-linear-memory.assemblyscript.en-us>
- Part #1: Memory management in WASM | by Divya Mohan - Dev Genius, <https://blog.devgenius.io/part-1-memory-management-in-wasm-52195f9b707f>
- Reconnecting mid-game | Netcode for GameObjects | 2.4.4 - Unity - Manual, <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.4/manual/advanced-topics/reconnecting-mid-game.html>
- Netcode load data and sync for late join players : r/Unity3D - Reddit, [https://www.reddit.com/r/Unity3D/comments/194tddf/netcode\\_load\\_data\\_and\\_sync\\_for\\_late\\_join\\_players/](https://www.reddit.com/r/Unity3D/comments/194tddf/netcode_load_data_and_sync_for_late_join_players/)
- How to implement synchronization of browser-based online games when users refresh their browser - Stack Overflow, <https://stackoverflow.com/questions/66639964/how-to-implement-synchronization-of-browser-based-online-games-when-users-refresh>
- Building Real-Time P2P Multiplayer Games in the Browser: Why I Eliminated the Server, <https://medium.com/@aguiran/building-real-time-p2p-multiplayer-games-in-the-browser-why-i-eliminated-the-server-d9f4ea7d4099>
- Wasm 3.0 Completed - WebAssembly, <https://webassembly.org/news/2025-09-17-wasm-3.0/>
- 24 WebAssembly-based Delta Sync for Cloud Storage Services - Group, <https://www.thucloud.com/zhenhua/papers/TOS'22%20WebAssembly%20Delta%20Sync.pdf>
- Evaluation of Delta Compression Techniques for Efficient Live Migration of Large Virtual Machines - ResearchGate,

[https://www.researchgate.net/publication/221137843\\_Evaluation\\_of\\_Delta\\_Compression\\_Techniques\\_for\\_Efficient\\_Live\\_Migration\\_of\\_Large\\_Virtual\\_Machines](https://www.researchgate.net/publication/221137843_Evaluation_of_Delta_Compression_Techniques_for_Efficient_Live_Migration_of_Large_Virtual_Machines)

- \*It is then further optimized with `wasm-opt -Oz --enable-bulk-memory` bringing ... | Hacker News, <https://news.ycombinator.com/item?id=33916614>
- CWASI: A WebAssembly Runtime Shim for Inter-function Communication in the Serverless Edge-Cloud Continuum - arXiv, <https://arxiv.org/html/2504.21503v1>
- A compiled-reactive language for high-performance WASM apps : r/WebAssembly - Reddit, [https://www.reddit.com/r/WebAssembly/comments/1qiaoez/a\\_compiledreactive\\_language\\_for\\_highperformance/](https://www.reddit.com/r/WebAssembly/comments/1qiaoez/a_compiledreactive_language_for_highperformance/)
- Advanced WebAssembly Performance Optimization: Pushing the Limits of Web Performance, <https://dev.to/rikinptl/advanced-webassembly-performance-optimization-pushing-the-limits-of-web-performance-4ke0>
- WebRTC Architecture (The WebRTC Series — Part 1) | by Bharath Kotha | Medium, <https://bharathkotha.medium.com/webrtc-architecture-the-webrtc-series-part-1-c1f21802ce32>
- [2603.24167] Walma: Learning to See Memory Corruption in WebAssembly - arXiv, <https://arxiv.org/abs/2603.24167>
- Multiplayer - WASM-4, <https://wasm4.org/docs/guides/multiplayer/>
- How 2XKO Handles Online Play & Netcode - Riot Games, <https://2xko.riotgames.com/en-gb/news/dev/how-2xko-handles-online-play/>
- 2024 WebRTC in Open Source Review: A Quantitative Analysis - webrtcHacks, <https://webrtchacks.com/2024-webrtc-in-open-source-review-a-quantitative-analysis/>